

A fair game of Rock-Paper-Scissors on (a public) blockchain

The challenge

Write a smart contract that allows two players to play “rock, paper, scissors” according to the following rules:

1) Create a game

One of the players starts a new game by calling the ``createGame`` method and sending a pot in ETH

2) Join the game

The second player agrees to play by calling the ``joinGame`` function and sending the same amount of ETH

3) Moves

At this point, both players can submit their moves (using the `move` function)

4) Result

Once both moves have been submitted, the smart contract calculates the result of the game and stores the distribution of the prize pool in the `pendingWithdrawals` mapping



Start from the solidity file `01-SillyRPS_start.sol`

Pull payments: why ?

Settlement should NOT push ether to the winner. Instead, it should credit `pendingWithdrawals[winner]`; the winner calls `withdraw()` to collect. Three benefits:

Bounded gas per settlement

The losing player cannot drag down settlement; `_settle` only writes storage and returns.

Hostile contract winners

If the winner is a contract whose `receive()` reverts, only that winner is affected — the game still settles.

Safe by CEI

`withdraw()` should zeroes the balance BEFORE the external call. A re-entrant `withdraw` sees 0 and reverts at `NothingToWithdraw`.

No ReentrancyGuard

Adding the guard would cost ~6 000 extra gas per call with no security benefit on this contract.



Start from the solidity file `01-SillyRPS_start.sol`

Problem: Every Ethereum transaction is public

On a public chain, a signed transaction first lands in the **mempool** — a shared pool that every node and every searcher can read. It typically stays there for several seconds before a block builder picks it up. During that window, its calldata is fully decoded and observable. Moreover, a **transaction** in a block is **public**

Mempool latency

Seconds (Ethereum mainnet) to tens of seconds during congestion. Plenty of time for a bot to react.

Calldata is plain

Function selector + arguments are visible. No HTTPS-style envelope encryption.

Transactions are public

All transactions contained in blocks are public, so, the second player can read the choice of the first player

MEV ecosystem

Specialised bots scan the mempool full-time and extract value via re-ordering, sandwiching, and back-running.

<https://sepolia.etherscan.io/tx/0xf0b91686ceb79c26f70083a4694325a743e70d37dfca77429de0ec605a6ebfea>

What a fair protocol must guarantee

Properties — borrowed from cryptography — pin down what "fair" means in this context. Every design decision in the rest of the deck flows from these three lines.

Hiding (privacy)

Neither player learns the other's move until both have locked themselves in.

Binding (no last-minute switch)

Once a player has committed, they cannot change their move retroactively.

Liveness (no stuck funds)

A player who refuses to cooperate cannot freeze the other's stake forever; the protocol settles in bounded time.

Trustless settlement

The outcome is determined by the contract, not by either player or any third party.

The idea: Commit/reveal

Cryptographic commitment, in one definition

A commitment scheme is a two-phase protocol between a sender and a receiver. In the **commit** phase the sender produces a value c that fixes a secret m ; in the **reveal** phase the sender opens c and shows that it indeed corresponds to m . Two properties:

Hiding

Given only c , the receiver learns nothing computationally useful about m . Without this property the sender's secret leaks during the commit phase.

Binding

After publishing c , the sender cannot produce a valid opening for any $m' \neq m$. Without this property the sender can equivocate after seeing the opponent's move.

The simplest commitment — a cryptographic hash

Keccak256: used carefully it satisfies BOTH properties of a commitment scheme and costs only $30 + 6 \cdot \lceil \text{len}/32 \rceil$ gas to compute.

Hiding from keccak256

Pre-image resistance: given $H(x)$ it is computationally infeasible to recover x — provided x has enough entropy (high-entropy input is the catch).

Binding from keccak256

Collision resistance: finding any $x' \neq x$ such that $H(x') = H(x)$ is computationally infeasible ($\approx 2^{128}$ work for a 256-bit output).

Cheap on chain

keccak256 is a precompile (opcode SHA3). About 36 gas for a single 32-byte word; trivial compared to a storage write.

Universal off chain

Implementations exist in every wallet, in ethers.js, solidity, ...

First attempt: $\text{commit} = H(\text{move})$ — and why it fails

If players commit to $H(\text{move})$ alone, the commitment is NOT hiding — the move space has only three elements. An attacker simply pre-computes $H(\text{Rock})$, $H(\text{Paper})$, $H(\text{Scissors})$ and looks the hash up in a 3-entry table.

```
// Brute-force "attack" — microseconds on a laptop
H(1) = keccak256(0x01) = 0x5fe7f9...   ← Rock
H(2) = keccak256(0x02) = 0x2eaa2b...   ← Paper
H(3) = keccak256(0x03) = 0x1bc9d1...   ← Scissors

observed commit = 0x2eaa2b... → the player committed to Paper.
// Hiding property is gone; the second player counters with Scissors.
```

Fix #1 — add a 256-bit random salt

Inflate the pre-image space by concatenating a uniformly random 32-byte salt. The hash now hides the move information-theoretically (as long as the salt is kept secret until the reveal).

```
commit = keccak256(abi.encodePacked(
    uint8(move),    // 1 byte:  Rock=1, Paper=2, Scissors=3
    salt            // 32 bytes: random — ethers.randomBytes(32)
));

// Pre-image search space:  $3 \cdot 2^{256}$ . Computationally infeasible.
// Salt MUST never appear on chain before the reveal phase.
```

Fix #2 — bind the commit to the player's address

Salt alone is still not enough. Player 2 can COPY player 1's commit hash bit-for-bit. P2 does not know the secret yet — but they don't need to: as soon as P1 commit, P2 replays the exact same and the contract accepts it. Result: forced draw, split pot.

```
// The fix
commit = keccak256(abi.encodePacked(
    uint8(move), salt, msg.sender
));
// Now P2 cannot replay P1's commit: the addresses differ
// → recomputed hash differs → reveal reverts with CommitMismatch.
```

```
echo "0x$(openssl rand -hex 32)"
```

Two-phase protocol

Both phases are explicit on-chain transactions. The salt is generated and stored client-side; only its hash crosses the network until reveal time.

Phase 1 — Commit

Each player publishes $c = H(\text{move} \parallel \text{salt} \parallel \text{player})$. Calldata reveals nothing about the move. Wallets and bots see only the hash.

Phase 2 — Reveal

Each player publishes (move, salt). Contract recomputes c' and accepts the reveal iff $c' == c$.

Settlement

Once both reveals are in, the contract decides the winner deterministically (cyclic rule) and credits the pot.



Start from the solidity file `02-FairRPS_start.sol`

Two-phase protocol: adding timeouts for liveness

Both phases are explicit on-chain transactions. The salt is generated and stored client-side (e.g. in localStorage); only its hash crosses the network until reveal time.

Phase 1 — Commit

Each player publishes $c = H(\text{move} \parallel \text{salt} \parallel \text{player})$. Calldata reveals nothing about the move. Wallets and bots see only the hash.

Phase 2 — Reveal

Each player publishes (move, salt). Contract recomputes c' and accepts the reveal iff $c' == c$.

Timeouts

Each phase has a deadline. A non-cooperator can be timed out so funds are not stuck.

Settlement

Once both reveals are in, the contract decides the winner deterministically (cyclic rule) and credits the pot.



Start from the solidity file 03-TimedFairRPS_start.sol

Adding timeouts

Define two different time windows: one for concluding commits and the other for the reveal phase

```
/// Two simple time windows. Choose values that comfortably exceed a  
/// transaction confirmation window – one hour is generous on Sepolia  
/// and even on mainnet during congestion.  
uint64 public constant COMMIT_WINDOW = 1 hours;  
uint64 public constant REVEAL_WINDOW = 1 hours;
```

Timeouts — liveness without trust

Anyone — not just the players — can drive the timeout once `block.timestamp` exceeds the deadline. The contract decides the payout based on who showed up.

Both committed / revealed

Phase has already transitioned — defensive revert. Should never happen.

One acted, the other did not

The cooperator takes the full pot ($2 \times \text{stake}$).

Neither acted

Refund each player their original stake (draw-like outcome with no winner).

`block.timestamp` may be manipulated by validators within a few seconds — completely irrelevant for `COMMIT_WINDOW = REVEAL_WINDOW = 1 hour`. No oracle needed.

Alice vs Bob — the full trace

t0	Alice → createGame{value: 0.01 ETH}()	returns gameId = 0
t1	Bob → joinGame(0){value: 0.01 ETH}	phase = Committing
t2	Alice computes commitA = H(Paper saltA alice) off-chain	
t3	Alice → commit(0, commitA)	
t4	Bob computes commitB = H(Scissors saltB bob) off-chain	
t5	Bob → commit(0, commitB)	phase = Revealing
t6	Alice → reveal(0, Paper, saltA)	reveal1 = Paper
t7	Bob → reveal(0, Scissors, saltB)	reveal2 = Scissors → _settle runs
t8	Settled(gameId=0, winner=Bob, payout=0.02 ETH)	phase = Settled
t9	Bob → withdraw()	Bob receives 0.02 ETH

Attacks and defences

Mempool sniffing of the move	Commit publishes only $H(\text{move} \parallel \text{salt} \parallel \text{player})$ — pre-image hidden.
Brute-force of the 3-move space	256-bit salt $\rightarrow$ pre-image space $3 \cdot 2^{256}$.
Commit-copying by player 2	msg.sender is hashed in; replay from another address fails the hash check.
Refusing to reveal	claimRevealTimeout awards the pot to the revealer.
Refusing to commit	Symmetric: claimCommitTimeout.
Re-entry on payout	CEI: balance zeroed before the external call.
Hostile contract winner blocking payout	Pull-payment: only that winner suffers, the game still settles.
Self-play (Alice plays herself)	joinGame reverts with SelfPlay.
Out-of-range move value	Solidity enum check + explicit Move.None rejection.
Validator-controlled timestamp skew	Windows of 1 hour; skew is at most a few seconds.

reveal — phase 2

Each player publishes (move, salt). The contract recomputes the expected hash with the SAME encoding it documented in the commit phase — including msg.sender. A mismatch reverts with CommitMismatch. Once both reveals are in, _settle runs.

```
function reveal(uint256 gameId, Move move, bytes32 salt) external {  
    Game storage g = games[gameId];  
    if (g.phase != Phase.Revealing) revert InvalidPhase();  
    if (move == Move.None) revert InvalidMove();  
    bytes32 expected =  
        keccak256(abi.encodePacked(uint8(move), salt, msg.sender));  
    /* match against commit1 or commit2; revert CommitMismatch if not */  
    if (g.reveal1 != Move.None && g.reveal2 != Move.None) _settle(gameId);  
}
```

RockPaperScissors.sol

Settlement — the cyclic R-P-S rule

Three pairs of (a, b) where a beats b: (Rock, Scissors), (Paper, Rock), (Scissors, Paper). Anything else with a == b is a draw. The pot — twice the stake — goes to the winner via pull-payment; a draw refunds each player their own stake.

```
function _settle(uint256 gameId) internal {
    Game storage g = games[gameId];
    address winner = _winner(g.reveal1, g.reveal2, g.player1, g.player2);
    uint256 pot     = g.stake * 2;
    if (winner == address(0)) {                // draw
        pendingWithdrawals[g.player1] += g.stake;
        pendingWithdrawals[g.player2] += g.stake;
    } else {                                    // winner takes pot
        pendingWithdrawals[winner] += pot;
    }
    g.phase = Phase.Settled;
    emit Settled(gameId, winner, winner == address(0) ? 0 : pot);
}
```

withdraw — Checks-Effects-Interactions in 10 lines

```
function withdraw() external {  
    uint256 amount = pendingWithdrawals[msg.sender];  
    if (amount == 0) revert NothingToWithdraw(); // CHECK  
  
    pendingWithdrawals[msg.sender] = 0; // EFFECT — must be first!  
  
    (bool ok, ) = msg.sender.call{value: amount}(""); // INTERACTION — last  
    if (!ok) revert TransferFailed();  
    emit Withdrawn(msg.sender, amount);  
}
```

RockPaperScissors.sol

A re-entrant withdraw from inside the recipient's receive() reads pendingWithdrawals[msg.sender] = 0 → reverts at the first line. The bug from The DAO is structurally impossible here.

PART V

Playing the game

Off-chain hash building; full Alice-vs-Bob trace

Off-chain — building the commit hash in ethers.js

The salt must be generated and stored CLIENT-SIDE. It never appears on chain until the reveal. ethers.solidityPackedKeccak256 reproduces Solidity's abi.encodePacked encoding byte-for-byte.

```
const ROCK = 1, PAPER = 2, SCISSORS = 3;
const move = PAPER;
const salt = ethers.hexlify(ethers.randomBytes(32)); // 32 random bytes

const commitHash = ethers.solidityPackedKeccak256(
  ["uint8", "bytes32", "address"],
  [move,      salt,      alice.address]
);
localStorage.setItem("rps_" + gameId + "_salt", salt); // persist the salt!
```

Lab — Remix walkthrough

The contract is self-contained. Paste in Remix, compile with Solidity 0.8.20+, deploy to Sepolia via MetaMask. Two browser windows (or two MetaMask accounts) emulate Alice and Bob.

1. Compile RockPaperScissors.sol (Solidity 0.8.20+)
2. Deploy with the Injected Provider — MetaMask on Sepolia
3. Account A: createGame VALUE = 0.01 SEP → note the gameId in the receipt
4. Account B: joinGame(0) VALUE = 0.01 SEP
5. Use makeCommitment(move, salt, you) — a pure function — to compute commit hashes
6. Each player commits, then each player reveals
7. Winner calls withdraw — pot lands in MetaMask

PART VI

Security analysis

What we defeat — and what remains open

What this design does NOT do

No griefing protection

A user can deliberately let the commit deadline elapse to make the opponent wait an hour. Stakes are returned, but time is wasted. Solvable with shorter timers + reputation.

No salt recovery

Lose the salt → you cannot reveal → you lose by timeout. A real dApp must persist salts in `localStorage` or derive them from a wallet-signed deterministic seed.

No lobby / matchmaking

Players need to share `gameId` off-chain. A full dApp would expose an `openGameIds[]` array or rely on event indexing.

No house fee / treasury

Winner takes 100% of the pot. Real money games typically skim 1–5 % to a protocol treasury — straightforward to add.

PART VII

Extensions & references

Where to go next, and where to read further

Possible extensions (good exam material)

Best-of-N rounds

Track roundsWon[player]; players re-commit per round.

ERC-20 stakes

Replace msg.value with safeTransferFrom; require allowance pre-game.

House fee

Skim e.g. 1 % of each pot to a treasury before paying the winner.

Searchable lobby

openGamelds[] + GameOpened event with the stake amount.

EIP-712-signed commits

Players sign typed messages; a relayer submits commits — gasless gameplay.

Chainlink VRF fallback

Random winner if both players time out on reveal.

Lizard-Spock variant

Five-move version; cyclic comparison generalises naturally.

N-player tournament

Bracket built on top of pairwise games + a check-in deadline.

References

- [1] Manuel Blum — Coin Flipping by Telephone (1983)
<https://dl.acm.org/doi/10.1145/1008908.1008911>
- [2] Pedersen — Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing (1992)
https://link.springer.com/chapter/10.1007/3-540-46766-1_9
- [3] Solidity — abi.encodePacked and keccak256 reference
<https://docs.soliditylang.org/en/latest/units-and-global-variables.html>
- [4] Solidity — Checks-Effects-Interactions pattern
<https://docs.soliditylang.org/en/latest/security-considerations.html>
- [5] Ethereum — MEV and the public mempool
<https://ethereum.org/en/developers/docs/mev/>
- [6] The DAO attack — re-entrancy post-mortem (June 2016)
<https://www.coindesk.com/learn/understanding-the-dao-attack/>
- [7] ethers.js v6 — solidityPackedKeccak256 reference
<https://docs.ethers.org/v6/api/hashing/>
- [8] EIP-712 — Typed structured data signing (for the gasless extension)
<https://eips.ethereum.org/EIPS/eip-712>
- [9] Chainlink VRF — Verifiable Random Function service
<https://docs.chain.link/vrf>
- [10] Course handout — RockPaperScissors.sol (this module)
<docs/ClaudeCode/commit-reveal-example/contract/RockPaperScissors.sol>

End of Case Study

Questions?

Source: docs/ClaudeCode/commit-reveal-example/contract/RockPaperScissors.sol